

Cloud-Based Self-Healing Infrastructure Monitoring System

Architecture Document

1. Overview

The Cloud-Based Self-Healing Infrastructure Monitoring System is designed to automatically detect system failures and recover from them without manual intervention. The system ensures high availability, scalability, and reliability by leveraging cloud-native services.

The system evolved from a basic infrastructure setup in Sprint 1 into an event-driven automated monitoring and recovery system in Sprint 2 using AWS services.

2. Evolution of Architecture:

Stage	Description	Key Additions
Sprint 1	Basic cloud infrastructure with EC2 instances, Load Balancer, Auto Scaling, and monitoring setup. Focused on deployment, traffic distribution, and system monitoring.	EC2 setup, Launch Template, ALB, Auto Scaling Group, CloudWatch monitoring
Sprint 2	Transition to event-driven architecture with automation, logging, and visualization. Added alerting, logging, and dashboard for system insights.	SNS notifications, Lambda logging, DynamoDB, CloudWatch Dashboard

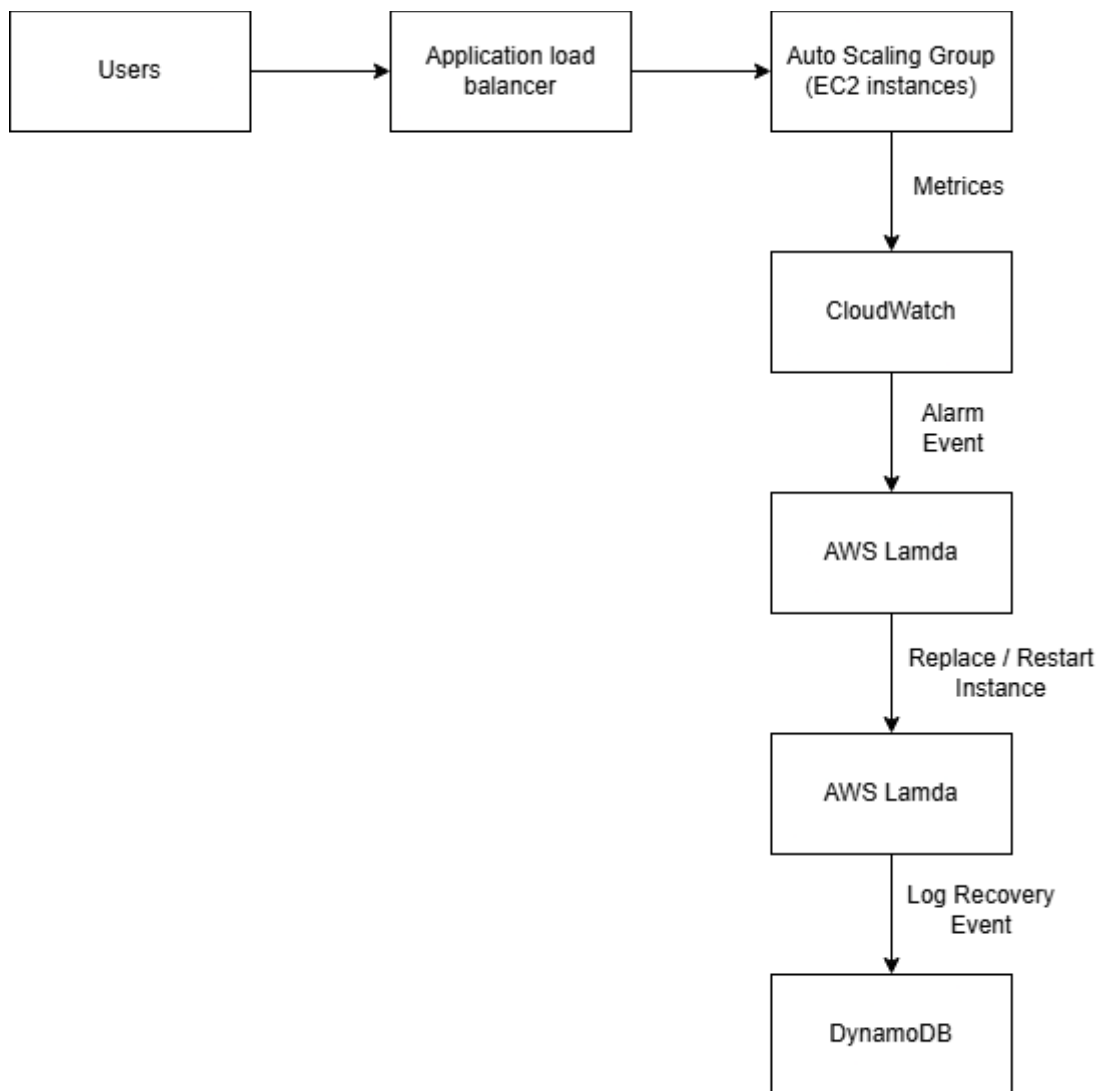
Why Event-Driven Architecture?

- System failures occur as real-time events (CPU spike, health check failure)
- CloudWatch generates alarm events
- Lambda reacts to events automatically
- Auto Scaling performs recovery without manual intervention

This approach:

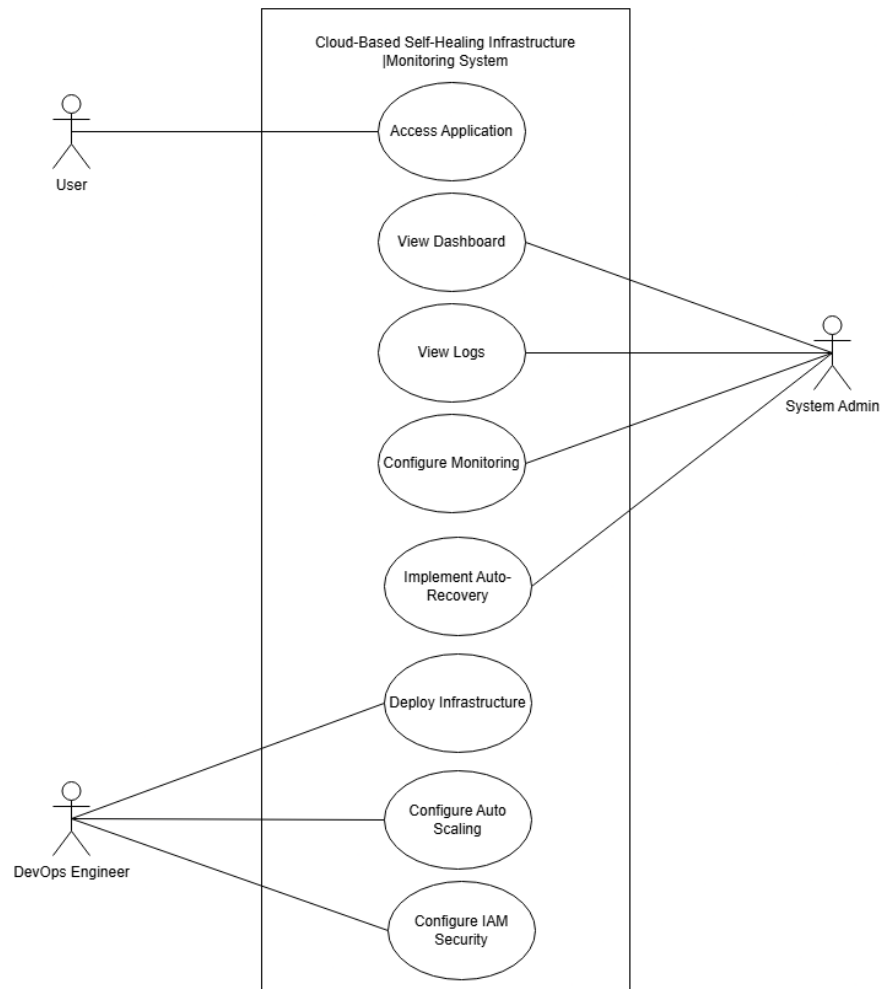
- Improves system reliability
- Reduces downtime
- Enables automatic recovery
- Supports scalable cloud infrastructure

High-Level Architecture:



- **Client:** Users access the system via a web browser through the Load Balancer
- **Compute Layer:** EC2 instances host the application
- **Scaling Layer:** Auto Scaling Group manages instance lifecycle
- **Monitoring Layer:** CloudWatch tracks metrics and triggers alarms
- **Alerting Layer:** SNS sends notifications
- **Automation Layer:** Lambda logs events
- **Storage Layer:** DynamoDB stores event logs
- **Visualization Layer:** CloudWatch Dashboard displays system metrics

3. Use Case Diagram



Actors:

- User
- System Administrator
- DevOps Engineer

Key Use Cases:

- Access Application
- Monitor System
- Receive Alerts
- View Dashboard
- Configure Infrastructure

4. Final System Architecture

4.1 Infrastructure Layer (Sprint 1)

1. EC2 instances are launched using Launch Templates.
2. Application Load Balancer distributes incoming traffic.
3. Auto Scaling Group ensures minimum instance availability.
4. Health checks ensure only healthy instances receive traffic.
5. CloudWatch monitors CPU utilization and system metrics.

4.2 Event-Driven Automation Layer (Sprint 2)

The system implements event-driven automation using AWS services:

1. CloudWatch detects anomalies (CPU spikes or failure).
2. CloudWatch Alarm triggers:
 - SNS → sends notification
 - Lambda → logs event
3. Lambda stores event details in DynamoDB.
4. Auto Scaling Group independently handles:
 - Scaling under load
 - Instance replacement on failure

4.3 Dashboard & Visualization Layer

1. CloudWatch Dashboard provides system insights.
2. Displays:
 - CPU Utilization
 - Request Count
 - Healthy Host Count
 - Instance Count
3. Helps visualize:
 - Load patterns
 - System health

- Scaling behavior

5. Database

5.1 Data Storage

The system uses DynamoDB as a NoSQL database for storing system events.

5.2 Schema Design

Table: RecoveryLogs

Attribute	Description
id	Unique identifier
timestamp	Event time
message	Event description

Purpose

- Store alarm-triggered events
- Track system behavior
- Provide audit trail for monitoring

6. Data Exchange Contract

6.1 Data Exchange Frequency

Data Entity	Frequency	Trigger
CPU Metrics	Every 1 minute	CloudWatch monitoring
Alarm Events	Event-based	Threshold breach
SNS Notification	Event-based	Alarm triggered
Lambda Execution	Event-based	Alarm triggered
DynamoDB Logs	Event-based	Lambda execution

6.2 Data Sets

- **Monitoring Data:** CPU usage, instance health
- **Alarm Data:** Threshold value, metric value
- **Log Data:** Timestamp, event message

6.3 Mode of Exchange

- EC2 → CloudWatch (metrics)
- CloudWatch → SNS (alert)
- CloudWatch → Lambda (trigger)
- Lambda → DynamoDB (storage)
- User → ALB → EC2 (HTTP request)

6.4 Frequency of Exchanges

- Real-time monitoring through CloudWatch
- Event-based triggering for alarms
- Instant logging via Lambda
- Continuous traffic routing via ALB